

1) **Objetivo da prototipação:** Validar ideias e funcionalidades antes da implementação final, permitindo ajustes rápidos com base em feedbacks. Pode ser aplicada nas fases de **concepção e elaboração**.

2) **Boas práticas de codificação:**

- **Código legível:** Facilita manutenção e colaboração.
- **Uso de comentários claros:** Ajuda outros desenvolvedores a entenderem o propósito do código.
- **Manter funções pequenas e específicas:** Facilita o teste e reutilização de código.

3) **Refatoração de código:** Melhorar a estrutura interna do código sem alterar seu comportamento. Benefícios: aumento de legibilidade e manutenção. Deve ser aplicada regularmente ou ao encontrar código confuso.

4) **Ferramentas de controle de versão (ex: Git):** Ajudam a acompanhar mudanças no código, facilitando colaboração e reversão de erros. Práticas recomendadas: commits frequentes, uso de branches e boas descrições de commits.

5) **Discordo:** Testes devem ocorrer em todas as fases do desenvolvimento, não apenas no fluxo de teste. Desenvolvedores podem realizar testes unitários enquanto desenvolvem.

6) **Importância da documentação:** Esclarece requisitos e decisões técnicas, facilitando a manutenção. Documentos essenciais incluem **manual de usuário** e **documentação de APIs**.

7) **Inspeção vs. Teste:** Inspeção revisa o código manualmente para encontrar erros; teste executa o código para verificar o comportamento.

8) **Concordo parcialmente:** Cobertura é importante, mas o foco deve ser encontrar erros críticos e garantir qualidade.

9) **Discordo:** Todos no time de desenvolvimento devem se envolver em testes, não apenas especialistas, pois testes podem ocorrer em diversos níveis.

10) **Exemplos de teste:**

- **Caixa branca:** Testar caminhos de execução de um algoritmo.
- **Caixa preta:** Testar a interface de um sistema sem conhecimento do código interno.

11) **Teste de aceitação vs. teste de unidade:**

- **Aceitação:** Valida requisitos do sistema. Exemplo: verificação de funcionalidades em um site.

- **Unidade:** Testa partes isoladas do código. Exemplo de sistema: validar uma API; integração: verificar comunicação entre dois módulos. **Regressão:** Verifica se mudanças no código não introduziram novos erros.

12) **Caso de teste:** Cenário que verifica uma funcionalidade específica. Exemplo: Validar login de usuário com credenciais corretas.

13) **Discordo:** Testes não são exaustivos, e existem métricas (cobertura, requisitos atendidos) que indicam quando parar.

14) **TDD:** Garante que o código seja escrito para passar em testes desde o início, promovendo design mais simples e maior confiança na qualidade. O processo envolve escrever testes antes de escrever o código.

15) **Desafios na fase de testes:**

- **Cobertura insuficiente:** Solução: priorizar testes de partes críticas.
- **Ambientes inconsistentes:** Solução: automação de testes em ambientes controlados.
- **Resistência a testes:** Solução: educação e cultura de qualidade.